



Banker's Algorithm in Operating System (OS) | Core Computer Science



Ujjwal Abhishek

What is the Banker's Algorithm?

The Banker's Algorithm developed by Edsger Wybe Dijkstra is a resource allocation and deadlock avoidance algorithm. It is also known as a deadlock detection algorithm.

The Banker's Algorithm is used to allocate resources to a process considering the availability of the resources and the predetermined maximum need of a process. It checks whether the resources can be allocated to a process or not, for its execution, which is called the “safe state” check.

Why it is named “Banker's Algorithm”?

It is named the “Banker's Algorithm” because the banks use the same technique to allocate money and sanction loans to their customers or account holders.

Suppose there are X number of account holders in a bank and the total sum of money they have deposited in the bank is M .

Before allocating any amount of money to a person the banks check if after subtracting that amount of money from the total money of the bank, the remaining money is



There are x account holders in the bank. The total sum of their money is M .

Now, Person Y demands the loan of amount Z .

Let the total amount of money in the bank is T .

if $(T - Z \geq M)$ then the loan will be sanctioned.

If $(T - Z < M)$ and the bank sanctions the loan then it will be an unsafe state as if all the customers demand their money at the same time then the bank would not be able to give them money.

Requirement of Banker's Algorithm

The Banker's Algorithm needs three things for its implementation.

1. Initial allocation
2. Maximum need
3. Total availability

By using these three things, the algorithm can allocate resources and can generate a safe sequence of execution for the processes.

Using the following data structures we can implement the Banker's Algorithm.

1. Let an array $Avail[]$ represents the currently available resources of each type.
2. A 2-d array $Max[][]$ represents the maximum need of each process where $Max[i][j]$ represents the maximum need of the j_{th} resource to the i_{th} process.
3. A 2-d array $Need[][]$ represents the current need of each process where $Need[i][j]$ represents the need of the j_{th} resource to the i_{th} process.
4. An array $Alloc[][]$ represents the currently allocated resources where $Alloc[i][j]$ represents the allocation of the j_{th} resource to the i_{th} process.
5. An array $Total[]$ represents the total available resources of each type before allocation.



2. Check if $\text{Need}[i][j] \leq A[j]$ then execute this process else check for the other process.
3. If $\text{Need}[i][j] \leq A[j]$, add $\text{Alloc}[i][j] + A[j]$ and the i_{th} process is terminated.
4. Repeat the 2nd step and 3rd step until all processes don't get executed.
5. After the successful execution of all processes, we get a safe sequence of execution.

Note: If at any point every processes' need is greater than available resources, it becomes an unsafe state. There can be more than one safe sequence of execution.

Example:

Let there are 5 processes P1, P2, P3, P4, P5, and 3 types of resources X, Y, Z.

Total resources X=7, Y=5, Z=7

Table 1.

Process	Maximum Need			Allocation		
	M[i][j]			A[i][j]		
	X	Y	Z	X	Y	Z
P1	6	5	3	1	0	2
P2	4	2	1	3	1	0
P3	5	1	2	0	1	1
P4	1	0	2	1	0	1
P5	5	2	3	1	0	2

Table 2.



$M[i][j] - A[i][j]$

	X	Y	Z
P1	5	5	1
P2	1	1	1
P3	5	0	1
P4	0	0	1
P5	4	2	1

Using the above algorithm we get the 2nd table from the 1st table.

Now, maintaining the current availability of the resources we get a safe sequence of execution for the above example which is $P2 \rightarrow P4 \rightarrow P5 \rightarrow P3 \rightarrow P1$.

Implementation of Banker's Algorithm in C++

We can implement the Banker's Algorithm using an array or a vector data structure in C++.

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n, m;
    n = 5; // Number of processes
    m = 3; // Number of resources

    vector<vector<int>> Max
    {
        { 6, 5, 3 },
        { 4, 2, 1 },
        { 5, 1, 2 },
        { 1, 0, 2 },
        { 5, 2, 3 }
```



```

        { 1, 0, 2 },
        { 3, 1, 0 },
        { 0, 1, 1 },
        { 1, 0, 1 },
        { 1, 0, 2 }
    };

    vector<int> Total { 7, 5, 7 }; // Total initial available resources
    vector<int> Avail(m); // Available resources after allocation
    for (int j = 0; j < m; j++)
    {
        int s = 0;
        for (int i = 0; i < n; i++)
        {
            s += Alloc[i][j];
        }
        Avail[j] = Total[j] - s;
    }

    vector<int> Exec(n, 0);
    vector<vector<int>> Need( n , vector<int> (m));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
            Need[i][j] = Max[i][j] - Alloc[i][j];
    }
    int k, c = 0;
    vector<int> seq;
    while (c < n)
    {
        for (int i = 0; i < n; i++)
        {
            if (Exec[i] == 0)
            {

```



```

        if (Need[i][j] > Avail[j])
        {
            mark = 1;
            break;
        }
    }
    if (mark == 0)
    {
        seq.push_back(i + 1);
        for (k = 0; k < m; k++)
        {
            Avail[k] += Alloc[i][k];
        }
        Exec[i] = 1;
    }
}

}

c++;
}

cout << "The safe sequence of execution is ";
for (int i = 0; i < n ; i++)
{
    if (i == n - 1)
        cout << " P" << seq[i] << "\n";
    else
        cout << " P" << seq[i] << " ->";
}
}

```

Output :

The safe sequence of execution is P2 -> P4 -> P5 -> P3 -> P1



Ujjwal Abhishek

Ujjwal is final-year CSE Undergraduate, a competitive coder, and a web developer passionate about problem-solving and data structures and algorithms.

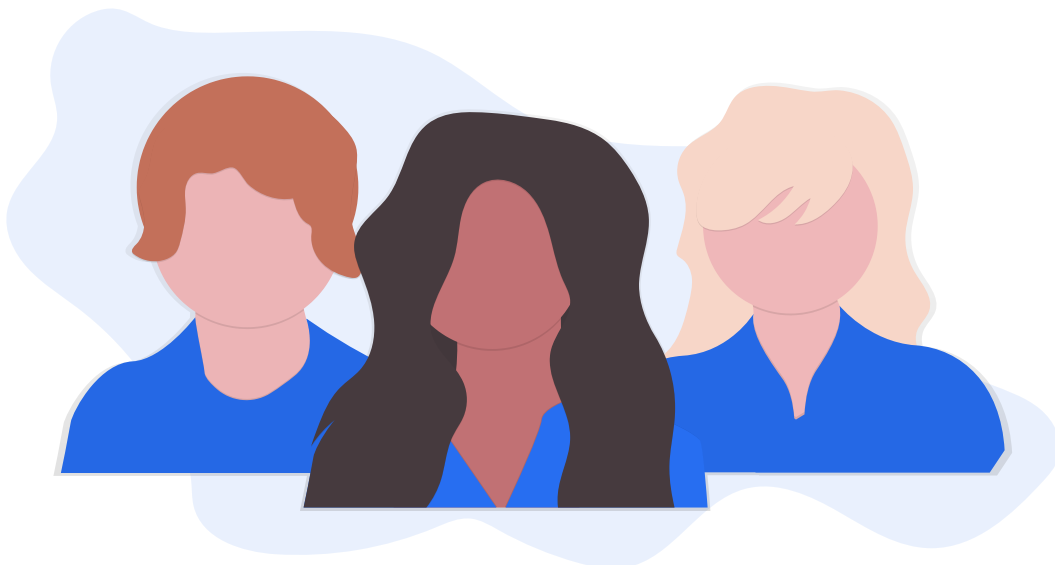
in



Practice Data Structures & Algorithms



Interview Prep Resources



Join our community

[About](#)

[FAQs](#)

[Terms](#)

[Privacy](#)

✉ hi@workat.tech





Data Structures & Algorithms

Machine Coding Round (LLD)

System Design & Architecture

Backend Development

Frontend Development

Awesome Project Ideas

Core Computer Science

Practice Questions

Machine Coding (LLD) Questions

System Design (HLD) Questions

Topic-wise DSA Questions

Company-wise DSA Questions

DSA Sheets (Curated Lists)

JavaScript Interview Questions

Frontend UI Machine Coding Questions

Online Compilers (IDE)

Online Java Compiler

Online C++ Compiler

Online C Compiler

Online Python Compiler

Online JavaScript Compiler

Topic-wise Problems

Dynamic Programming Interview Questions

Linked List Interview Questions

Graph Interview Questions

Backtracking Interview Questions

Arrays Interview Questions

Trees Interview Questions

Company-wise Problems



[Flipkart Interview Questions](#)

[Adobe Interview Questions](#)

[Facebook Interview Questions](#)

DSA Sheets (Curated Lists)

[Top Interview Questions](#)

[FAANG Interview Questions](#)

[Most Asked Interview Questions](#)

[6 month DSA Practice Sheet](#)

[3 month DSA Practice Sheet](#)

[Last minute DSA Practice Sheet](#)

© 2020-2022 WORKATTECH TECHNOLOGIES PVT LTD
ALL RIGHTS RESERVED